

FCFS:

```
#include <iostream>
```

```
using namespace std;
```

```
int main() {
```

```
    int n;
```

```
    cout << "Enter number of processes: ";
```

```
    cin >> n;
```

```
    // Arrays for process data
```

```
    int pid[10], at[10], bt[10], ct[10], tat[10], wt[10];
```

```
    // Variables to store total waiting time and turnaround time
```

```
    int total_wt = 0, total_tat = 0;
```

```
    // Input
```

```
    for(int i = 0; i < n; i++) {
```

```
        pid[i] = i + 1;
```

```
        cout << "\nEnter Arrival Time for Process " << pid[i] << ": ";
```

```
        cin >> at[i];
```

```
        cout << "Enter Burst Time for Process " << pid[i] << ": ";
```

```
        cin >> bt[i];
```

```
    }
```

```
    // Calculate Completion Time (NO SORTING - input order maintained)
```

```
    int current_time = 0;
```

```
    for(int i = 0; i < n; i++) {
```

```
        if(current_time < at[i]) {
```

```
            current_time = at[i]; // CPU idle, wait until arrival
```

```
        }
```

```
        ct[i] = current_time + bt[i];
```

```
        current_time = ct[i];
```

```
    }
```

```
    // Calculate TAT and WT
```

```
    for(int i = 0; i < n; i++) {
```

```
        tat[i] = ct[i] - at[i];
```

```
        wt[i] = tat[i] - bt[i];
```

```
    // Add to total for average calculation later
```

```
    total_wt += wt[i];
```

```
    total_tat += tat[i];
```

```
    }
```

```
    // Output Table
```

```
    cout << "\nProcess\tAT\tBT\tCT\tTAT\tWT\n";
```

```
    for(int i = 0; i < n; i++) {
```

```
        cout << "P" << pid[i] << "\t"
```

```
        << at[i] << "\t"
```

```
        << bt[i] << "\t"
```

```
        << ct[i] << "\t"
```

```
        << tat[i] << "\t"
```

```
        << wt[i] << endl;
```

```
    }
```

```
    // --- Gantt Chart with Idle Time Logic ---
```

```

cout << "\nGantt Chart:\n";

int time = 0; // Start from time 0

// First, print the starting time (usually 0)
cout << time;

for(int i = 0; i < n; i++) {
    // Check if there is an idle period before this process starts
    if(time < at[i]) {
        // Print Idle period
        cout << " -- Idle -- " << at[i];
        time = at[i]; // Update time to arrival time
    }

    // Print the Process
    cout << " -- P" << pid[i] << " -- " << ct[i];

    // Update time to completion time of current process
    time = ct[i];
}
cout << endl;

// --- Average Calculations ---
cout << "\nAverage Waiting Time: "
    << total_wt / n << endl;

cout << "Average Turnaround Time: "
    << total_tat / n << endl;

return 0;
}

```

□ স্যারকে যা বলবে (Step-by-Step Explanation):

১. ভূমিকা (Introduction)

"স্যার, এই কোডটি FCFS (First-Come, First-Served) শেডিউলিং অ্যালগরিদম ইমপ্লিমেন্ট করে। FCFS হলো সবচেয়ে সহজ শেডিউলিং অ্যালগরিদম, যেখানে প্রসেসগুলো তাদের আগমনের ক্রম (Arrival Order) অনুযায়ী CPU পায়। একবার একটি প্রসেস CPU পেলে, সেটি সম্পূর্ণ শেষ না হওয়া পর্যন্ত CPU ছাড়বে না (Non-Preemptive)।"

২. ইনপুট নেওয়া (Input Section)

"প্রথমে ইউজার থেকে প্রসেসের সংখ্যা n ইনপুট নেওয়া হয়েছে। এরপর একটি লুপ চালিয়ে প্রতিটি প্রসেসের Arrival Time (AT) এবং Burst Time (BT) নেওয়া হয়েছে। প্রসেসের নাম $P_1, P_2 \dots$ হিসেবে $pid[]$ অ্যারেতে সেভ করা হয়েছে।"

৩. Completion Time (CT) হিসাব করা (Main Logic)

"এরপর একটি for লুপ চালিয়ে প্রতিটি প্রসেসের Completion Time (CT) বের করা হয়েছে।"

- লুপের শুরুতে চেক করা হয়: `if(current_time < at[i])`।
- যদি বর্তমান সময় প্রসেসের আগমন সময়ের চেয়ে কম হয়, তার মানে CPU খালি পড়ে আছে। তখন `current_time` কে প্রসেসের `at[i]` এর সমান করে দেওয়া হয় (অর্থাৎ CPU ওই প্রসেস আসা পর্যন্ত অপেক্ষা করে বা Idle থাকে)।
- এরপর `ct[i] = current_time + bt[i]` সূত্র দিয়ে প্রসেসটির শেষ হওয়ার সময় বের করা হয় এবং `current_time` আপডেট করা হয়।"

৪. TAT এবং WT হিসাব করা

"CT বের করার পর, আরেকটি লুপ চালিয়ে Turnaround Time (TAT) এবং Waiting Time (WT) বের করা হয়েছে।"

- `TAT = CT - AT`
- `WT = TAT - BT`
- পাশাপাশি, গড় মান বের করার জন্য মোট WT এবং TAT যোগ করে `total_wt` এবং `total_tat` ভেরিয়েবলে জমা করা হয়েছে।"

৫. গ্যান্ট চার্ট প্রিন্টিং (Gantt Chart with Idle Handling)

"গ্যান্ট চার্টটি সুন্দরভাবে দেখানোর জন্য একটি আলাদা লুপ ব্যবহার করা হয়েছে।"

- শুরুতে সময় 0 থেকে শুরু করা হয়।
- প্রতিটি প্রসেসের আগে চেক করা হয়: `if(time < at[i])`। যদি সত্য হয়, তবে গ্যান্ট চার্টে `-- Idle --` প্রিন্ট করা হয় এবং সময়কে প্রসেসের আগমন সময়ের সমান করা হয়।
- এরপর প্রসেসের নাম (P1, P2...) এবং তার শেষ সময় (`ct[i]`) প্রিন্ট করা হয়।
- এতে করে গ্যান্ট চার্টে ফাঁকা জায়গাগুলো (Idle Time) স্পষ্টভাবে দেখা যায়।"

৬. ফাইনাল আউটপুট (Average Calculation)

"শেষে, একটি টেবিলের মাধ্যমে প্রতিটি প্রসেসের বিস্তারিত তথ্য (AT, BT, CT, TAT, WT) দেখানো হয়েছে। এরপর মোট WT এবং TAT কে প্রসেস সংখ্যা `n` দিয়ে ভাগ করে গড় ওয়েটিং টাইম এবং গড় টার্নঅরাউন্ড টাইম প্রিন্ট করা হয়েছে।"

⚡ স্যারের সম্ভাব্য প্রশ্নের উত্তর (Quick Fire):

প্রশ্ন	উত্তর
এটি কোন অ্যালগরিদম?	এটি FCFS (First-Come, First-Served)।
এটি Preemptive নাকি Non-Preemptive?	এটি Non-Preemptive। একবার প্রসেস শুরু হলে শেষ পর্যন্ত চলে।
কোনো সটিং করা হয়েছে?	না, ইনপুটের ক্রম অনুযায়ীই প্রসেস নেওয়া হয়েছে। শুধু আগমন সময়ের সাথে বর্তমান সময়ের তুলনা করে Idle Time হ্যান্ডেল করা হয়েছে।
Idle Time কখন হয়?	যখন বর্তমান সময় (<code>current_time</code>) প্রসেসের আগমন সময় (<code>at[i]</code>) এর চেয়ে কম হয়, তখন CPU খালি থাকে।
TAT এবং WT এর সূত্র কী?	$TAT = CT - AT$; $WT = TAT - BT$ ।

✓শেষ কথা:

ভাই, এই কোডটি খুব সরল। স্যারকে বলার সময় "Idle Time হ্যান্ডেলিং" এবং "Non-Preemptive nature" এই দুটি বিষয় জোর দিয়ে।

```

..
#include <iostream> // হেডার ফাইল ইনক্লুড করা (ইনপুট/আউটপুটের জন্য)
using namespace std; // স্ট্যান্ডার্ড নেমস্পেস ব্যবহার করা

int main() {
    int n; // প্রসেসের সংখ্যা রাখার ভেরিয়েবল

    // ১. ইউজার থেকে প্রসেসের সংখ্যা ইনপুট নেওয়া
    cout << "Enter number of processes: ";
    cin >> n;

    // অ্যারে ডিক্লেয়ারেশন (সর্বোচ্চ ১০টি প্রসেস ধরা হয়েছে)
    int pid[10], at[10], bt[10], ct[10], tat[10], wt[10];
    // pid = Process ID, at = Arrival Time, bt = Burst Time
    // ct = Completion Time, tat = Turnaround Time, wt = Waiting Time

    // মোট ওয়েটিং টাইম এবং টার্নআরাউন্ড টাইম জমা রাখার ভেরিয়েবল
    int total_wt = 0, total_tat = 0;

    // ২. ইউজার থেকে প্রতিটি প্রসেসের AT এবং BT ইনপুট নেওয়া
    for(int i = 0; i < n; i++) {
        pid[i] = i + 1; // প্রসেসের নাম P1, P2... হিসেবে সেট করা

        cout << "\nEnter Arrival Time for Process " << pid[i] << ": ";
        cin >> at[i]; // অ্যারাইভাল টাইম ইনপুট

        cout << "Enter Burst Time for Process " << pid[i] << ": ";
    }
}

```

```

    cin >> bt[i]; // বাস্ট টাইম ইনপুট
}

// ৩. Completion Time (CT) হিসাব করার লুপ
int current_time = 0; // বর্তমান সময় শুরুতে ০

for(int i = 0; i < n; i++) {
    // চেক করা: বর্তমান সময় কি প্রসেসের আগমন সময়ের চেয়ে কম?
    if(current_time < at[i]) {
        current_time = at[i]; // যদি কম হয়, তবে CPU খালি থাকে (Idle), সময়কে AT এর সমান করা
    }

    // Completion Time = বর্তমান সময় + বাস্ট টাইম
    ct[i] = current_time + bt[i];

    // পরবর্তী প্রসেসের জন্য বর্তমান সময় আপডেট করা
    current_time = ct[i];
}

// ৪. TAT এবং WT হিসাব করার লুপ
for(int i = 0; i < n; i++) {
    // Turnaround Time = Completion Time - Arrival Time
    tat[i] = ct[i] - at[i];

    // Waiting Time = Turnaround Time - Burst Time
    wt[i] = tat[i] - bt[i];

    // গড় বের করার জন্য মোট WT এবং TAT যোগ করা
    total_wt += wt[i];
    total_tat += tat[i];
}

// ৫. আউটপুট টেবিল প্রিন্ট করা
cout << "\nProcess\tAT\tBT\tCT\tTAT\tWT\n"; // টেবিলের হেডার
cout << "-----\n";

for(int i = 0; i < n; i++) {
    // প্রতিটি প্রসেসের বিস্তারিত তথ্য টেবিল আকারে প্রিন্ট করা
    cout << "P" << pid[i] << "\t"
        << at[i] << "\t"
        << bt[i] << "\t"
        << ct[i] << "\t"
        << tat[i] << "\t"
        << wt[i] << endl;
}

// ৬. গ্যান্ট চার্ট প্রিন্ট করা (Idle Time সহ)
cout << "\nGantt Chart:\n";

int time = 0; // গ্যান্ট চার্টের জন্য সময় আবার ০ থেকে শুরু
cout << time; // শুরু সময় (০) প্রিন্ট করা

for(int i = 0; i < n; i++) {
    // চেক করা: বর্তমান সময় কি প্রসেসের আগমন সময়ের চেয়ে কম?
    if(time < at[i]) {

```

```

        // যদি কম হয়, তবে CPU খালি ছিল (Idle)
        cout << " -- Idle -- " << at[i]; // Idle এবং নতুন সময় প্রিন্ট করা
        time = at[i]; // সময়কে প্রসেসের আগমন সময়ের সমান করা
    }

    // প্রসেসের নাম এবং শেষ সময় প্রিন্ট করা
    cout << " -- P" << pid[i] << " -- " << ct[i];

    // সময়কে প্রসেসের শেষ হওয়ার সময়ের সমান করা
    time = ct[i];
}
cout << endl; // নতুন লাইন

// ৭. গড় ওয়েটিং টাইম এবং টার্নঅরাউন্ড টাইম প্রিন্ট করা
cout << "\nAverage Waiting Time: " << (float)total_wt / n << endl;
cout << "Average Turnaround Time: " << (float)total_tat / n << endl;
// (float) ব্যবহার করা হয়েছে যাতে দশমিক মান সঠিক আসে

return 0; // প্রোগ্রাম সফলভাবে শেষ
}

```

////////////////////////////////////

SJF:

```

#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    int at[10], bt[10], ct[10], tat[10], wt[10];
    bool done[10] = {false}; // কোন প্রসেস শেষ হয়েছে তা ট্র্যাক করার জন্য

    // Input
    for(int i = 0; i < n; i++) {
        cout << "\nEnter Arrival Time for P" << (i+1) << ": ";
        cin >> at[i];
        cout << "Enter Burst Time for P" << (i+1) << ": ";
        cin >> bt[i];
    }
}

```

```

int time = 0;    // বর্তমান সময়
int completed = 0; // কতগুলো প্রসেস শেষ হয়েছে
int total_wt = 0, total_tat = 0;

cout << "\nGantt Chart:\n";
cout << time; // শুরু সময় (সাধারণত 0)

while(completed < n) {
    int idx = -1;    // mne amr hate ekhono kno process nei,kujtci ekhono
    int min_bt = 100000; // shob theke kom BT khujar jnno like 5<10000 theke choto hoite hobe
    shob

    // রেডি কিউ থেকে shortest job খুঁজুন
    for(int i = 0; i < n; i++) {
        if(at[i] <= time && !done[i]) {
            if(bt[i] < min_bt) {
                min_bt = bt[i];
                idx = i;
            }
        }
    }

    if(idx != -1) {
        // প্রসেস এক্সিকিউট করুন
        cout << " -- P" << (idx+1) << " -- ";

        time += bt[idx];    // সময় বাড়ান
        ct[idx] = time;    // Completion Time
        tat[idx] = ct[idx] - at[idx]; // Turnaround Time
        wt[idx] = tat[idx] - bt[idx]; // Waiting Time

        total_wt += wt[idx];
        total_tat += tat[idx];

        done[idx] = true; // প্রসেস শেষ হিসেবে চিহ্নিত করুন
        completed++;

        cout << time;    // শেষ সময় প্রিন্ট করুন
    }
    else {
        // CPU Idle
        cout << " -- Idle -- ";
        time++;    // সময় ১ ইউনিট বাড়ান
        cout << time;    // নতুন সময় প্রিন্ট করুন
    }
}

// Output Table
cout << "\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n";
for(int i = 0; i < n; i++) {
    cout << "P" << (i+1) << "\t" << at[i] << "\t" << bt[i] << "\t"
        << ct[i] << "\t" << tat[i] << "\t" << wt[i] << endl;
}

// Average Times

```

```
cout << "\nAverage Waiting Time: " << (float)total_wt / n << endl;
cout << "Average Turnaround Time: " << (float)total_tat / n << endl;

return 0;
}
```

/////

□ স্যারকে যা বলবে (Step-by-Step Explanation):

১. শুরুতে (Initialization & Input)

"স্যার, প্রথমে আমি প্রসেসের সংখ্যা `n` ইনপুট নিয়েছি। এরপর প্রতিটি প্রসেসের Arrival Time (AT) এবং Burst Time (BT) অ্যাারেতে নিয়েছি। একটি `done[]` অ্যাারে রাখা হয়েছে যা ট্র্যাক করবে কোন প্রসেস শেষ হয়েছে আর কোনটি বাকি আছে।"

২. মেন লজিক (The While Loop & SJF Selection)

"এরপর একটি `while` লুপ চালানো হয়েছে যতক্ষণ না সব প্রসেস (`completed < n`) শেষ হয়। লুপের প্রতিবার রাউন্ডে, আমি একটি `for` লুপ চালিয়ে চেক করি: ১. কোন প্রসেসগুলো ইতিমধ্যে এসে গেছে (`AT <= current_time`)। ২. তাদের মধ্যে থেকে যার Burst Time সবচেয়ে কম, সেটিকে বেছে নেই। একেই Non-Preemptive SJF লজিক বলে।"

৩. Idle Time হ্যান্ডেলিং (Important Point!)

"যদি দেখা যায় যে বর্তমান সময়ে কোনো প্রসেস আসেনি (অর্থাৎ `idx == -1`), তখন CPU খালি থাকে। একে Idle Time বলে। তখন আমি শুধু সময় ১ ইউনিট বাড়িয়ে দিয়ে আবার চেক করি। এতে করে গ্যান্ট চার্টে ফাঁকা জায়গাগুলোও সঠিকভাবে দেখানো যায়।"

৪. এক্সিকিউশন ও ক্যালকুলেশন (Execution & Calculation)

"যখন একটি প্রসেস পাওয়া যায়, তখন:

- তার Burst Time অনুযায়ী `current_time` বাড়ানো হয়।
- Completion Time (CT), Turnaround Time ($TAT = CT - AT$), এবং Waiting Time ($WT = TAT - BT$) বের করা হয়।
- প্রসেসটিকে `done[]` অ্যাারেতে `true` করে দেওয়া হয় যাতে এটি আর না নেওয়া হয়।
- সবশেষে, মোট WT এবং TAT যোগ করে গড় মান বের করা হয়।"

⚡ ল্যাব এক্সামের জন্য "গোল্ডেন টিপস" (স্যারের প্রশ্নের উত্তর):

স্যার যদি মাঝপথে প্রশ্ন করেন, তবে এই ছোট উত্তরগুলো মনে রাখবে:

স্যারের প্রশ্ন	তোমার উত্তর (সহজ ভাষায়)
কেন <code>done[]</code> অ্যারে লাগল?	"স্যার, একবার যে প্রসেস শেষ হয়ে গেছে, যেন লুপে আবার সেটি না আসে, সেজন্য <code>done[]</code> ব্যবহার করেছি।"
কেন <code>min_bt = 100000</code> দিয়েছেন?	"স্যার, শুরুতে একটি বড় সংখ্যা ধরেছি যাতে প্রথম প্রসেসটির সাথে তুলনা করে সহজেই ছোট সংখ্যাটি খুঁজে পাওয়া যায়।"
Idle Time কেন লাগল?	"স্যার, যদি প্রসেসের Arrival Time বর্তমান সময়ের চেয়ে বেশি হয়, তবে CPU খালি পড়ে থাকে। তাই সময় বাড়তে Idle Time দরকার।"
(float) কেন ব্যবহার করেছেন?	"স্যার, গড় মান বের করার সময় যেন দশমিক অংশ কেটে না যায়, সেজন্য Integer কে Float-এ কাস্ট করে ভাগ করেছি।"
SJF কি Preemptive নাকি Non-Preemptive?	"স্যার, এই কোডটি Non-Preemptive কারণ একবার প্রসেস CPU পেলে, সেটি শেষ না হওয়া পর্যন্ত CPU ছাড়বে না।"

বোর্ড/খেতায় আঁকার জন্য ছোট ডায়াগ্রাম আইডিয়া:

স্যার যদি বলেন "লজিকটা দেখাও", তবে খাতায় এইভাবে আঁকতে পারো:

1. **Start:** `time = 0, completed = 0`
2. **Loop:** Check all processes where `AT <= time` and `!done`.
3. **Select:** Pick process with Min Burst Time.
4. **If Found:**
 - `time += BT`
 - Calculate CT, TAT, WT
 - Mark `done = true`
5. **If Not Found:**
 - `time++` (Idle)
6. **End:** When `completed == n`.

SRTE:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n = 5;
```

```

int at[10] = {3, 1, 4, 0, 2};
int bt[10] = {1, 4, 2, 6, 3};

int rt[10]; // Remaining Time track korar jnno
for(int i=0; i<n; i++) rt[i] = bt[i]; // surute Remaining Time = Burst Time

int ct[10], tat[10], wt[10];
bool completed[10] = {false};

int time = 0;
int completed_count = 0;
int total_wt = 0, total_tat = 0;

cout << "Gantt Chart:\n";
cout << time; // starting time

while(completed_count < n) {
    int idx = -1; // dhore neci surute kno process pawa jay nay
    int min_rt = 100000;

    // shob cheye kom remaining time er process kujtci
    for(int i=0; i<n; i++) {
        if(at[i] <= time && !completed[i]) { //procee ashce kintu not completed?
            if(rt[i] < min_rt) { //Remaining time ki ager min_time 10000 thekeo ki choto?kina
                min_rt = rt[i]; //choto hole notun choto RT mone rakbo
                idx = i;
            }
        }
    }

    if(idx != -1) { // mne process khuje paici

        //time barte thakte and remiang time kombe
        rt[idx]--;
        time++;

        // process jokhn sesh hye jay ekbare
        if(rt[idx] == 0) {
            completed[idx] = true;
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            total_wt += wt[idx];
            total_tat += tat[idx];

            completed_count++;
        }

    } else {
        // CPU Idle
        cout << " -- Idle -- " << (time + 1);
        time++;
    }
}

```

```

// part 2 eta hocce gantt chart akar jnno
for(int i=0; i<n; i++) rt[i] = bt[i];
for(int i=0; i<n; i++) completed[i] = false;
time = 0;
completed_count = 0;

cout << "\n\nFinal Gantt Chart:\n";
cout << "0";

int last_process = -1; // ager process track korar jnno

while(completed_count < n) {
    int idx = -1;
    int min_rt = 100000;

    for(int i=0; i<n; i++) {
        if(at[i] <= time && !completed[i]) {
            if(rt[i] < min_rt) {
                min_rt = rt[i];
                idx = i;
            }
        }
    }

    if(idx != -1) {

        if(last_process != idx) {
            cout << " -- P" << (idx+1);
            last_process = idx;
        }

        rt[idx]--;
        time++;

        if(rt[idx] == 0) {
            completed[idx] = true;
            completed_count++;
            cout << " -- " << time; // process sesh howar shomoy
            last_process = -1; // porer process jnno reset
        }
    } else {
        cout << " -- Idle -- " << (time + 1);
        time++;
        last_process = -1;
    }
}

// Output Table
cout << "\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n";
for(int i=0; i<n; i++) {
    cout << "P" << (i+1) << "\t" << at[i] << "\t" << bt[i] << "\t"
        << ct[i] << "\t" << tat[i] << "\t" << wt[i] << endl;
}

cout << "\nAverage Waiting Time: " << (float)total_wt / n << endl;
cout << "Average Turnaround Time: " << (float)total_tat / n << endl;

```

```
return 0;  
}
```

● পার্ট ১: ইনপুট এবং ভেরিয়েবল সেটআপ

cpp

```
1 #include <iostream>  
2 using namespace std;  
3  
4 int main() {  
5     int n = 5; // প্রসেসের সংখ্যা (হার্ডকোডেড, চাইলে cin >> n; করতে পারো)  
6  
7     // Arrival Time (AT) এবং Burst Time (BT) ইনপুট অ্যারে  
8     int at[10] = {3, 1, 4, 0, 2};  
9     int bt[10] = {1, 4, 2, 6, 3};  
10  
11     int rt[10]; // Remaining Time ট্র্যাক করার জন্য অ্যারে  
12     for(int i=0; i<n; i++) rt[i] = bt[i]; // শুরুতে RT = BT  
13  
14     int ct[10], tat[10], wt[10]; // ফলাফল রাখার অ্যারে  
15     bool completed[10] = {false}; // কোন প্রসেস শেষ হয়েছে তা ট্র্যাক করার জন্য  
16  
17     int time = 0; // বর্তমান সময় (Clock)  
18     int completed_count = 0; // কতগুলো প্রসেস শেষ হয়েছে  
19     int total_wt = 0, total_tat = 0; // গড় বের করার জন্য মোট যোগফল
```

📌 স্যারকে যা বলবে:

"স্যার, প্রথমে আমি প্রসেসের সংখ্যা `n` এবং তাদের Arrival Time (`at`) ও Burst Time (`bt`) নিয়েছি। SRTF যেহেতু Preemptive, তাই আমাদের ট্র্যাক করতে হবে প্রতিটি প্রসেসের কতটুকু কাজ বাকি আছে। সেজন্য একটি `rt[]` (Remaining Time) অ্যারে নিয়েছি, যা শুরুতে `bt` এর সমান। এছাড়া `completed[]` অ্যারে দিয়ে ট্র্যাক করছি কোন প্রসেস শেষ হয়েছে।"

// --- ১ম লুপ: CT, TAT, WT হিসাব করা ---

```
while(completed_count < n) {  
    int idx = -1; // কোন প্রসেস নেওয়া হবে  
    int min_rt = 100000; // সবচেয়ে কম Remaining Time খুঁজার জন্য
```

// Shortest Remaining Time খুঁজে বের করা

```
for(int i=0; i<n; i++) {  
    if(at[i] <= time && !completed[i]) { // শর্ত: প্রসেস এসেছে? এবং শেষ হয়নি?  
        if(rt[i] < min_rt) { // শর্ত: এর RT কি আগের সবচেয়ে ছোটটার চেয়েও ছোট?  
            min_rt = rt[i]; // হ্যাঁ হলে, নতুন সবচেয়ে ছোট RT মনে রাখো  
            idx = i; // এবং এই প্রসেসের নাম (index) মনে রাখো  
        }  
    }  
}
```

```
if(idx != -1) {  
    // প্রসেস পাওয়া গেছে
```

```
    rt[idx]--; // Remaining Time ১ ইউনিট কমানো (Preemption চেকের জন্য)  
    time++; // সময় ১ ইউনিট বাড়ানো
```

```

// যদি প্রসেসটি সম্পূর্ণ শেষ হয়ে যায়
if(rt[idx] == 0) {
    completed[idx] = true;    // প্রসেসকে 'শেষ' চিহ্নিত করো
    ct[idx] = time;           // Completion Time সেট করো
    tat[idx] = ct[idx] - at[idx]; // Turnaround Time বের করো
    wt[idx] = tat[idx] - bt[idx]; // Waiting Time বের করো

    total_wt += wt[idx];      // মোট WT যোগ করো
    total_tat += tat[idx];    // মোট TAT যোগ করো

    completed_count++;        // সম্পন্ন প্রসেসের কাউন্ট বাড়ো
}
} else {
    // CPU Idle (কোনো প্রসেস পাওয়া যায়নি)
    time++; // সময় ১ ইউনিট বাড়িয়ে আবার চেক করো
}
}
}

```

৯. স্যারকে যা বলবে:

"স্যার, এই `while` লুপটি ততক্ষণ চলবে যতক্ষণ না সব প্রসেস শেষ হয়।

১. প্রতিবার লুপে একটি `for` লুপ চালিয়ে দেখা হয় কোন প্রসেসগুলো এসে গেছে এবং তাদের মধ্যে কার **Remaining Time** সবচেয়ে কম। সেটিকে বেছে নেওয়া হয়।

২. যদি কোনো প্রসেস পাওয়া যায়, তবে তার `rt` ১ কমানো হয় এবং `time` ১ বাড়ানো হয়। এটি নিশ্চিত করে যে প্রতি ১ ইউনিট সময় পরপর আমরা চেক করছি নতুন কোনো ছোট প্রসেস এসেছে কিনা (Preemption)।

৩. যখন `rt` শূন্য হয়, তখন প্রসেসটি শেষ হয় এবং তার CT, TAT, WT বের করে মোট যোগফলে যুক্ত করা হয়।

৪. যদি কোনো প্রসেস না পাওয়া যায়, তবে CPU Idle থাকে এবং সময় বাড়ানো হয়।"

// --- ২য় লুপ: গ্যান্ট চার্ট প্রিন্ট করা ---

// ১. সবকিছু রিসেট করা (আবার শুরু থেকে সিমুলেশন করার জন্য)

```

for(int i=0; i<n; i++) rt[i] = bt[i];    // RT কে আবার BT এর সমান করো
for(int i=0; i<n; i++) completed[i] = false; // সব প্রসেসকে আবার 'অসম্পন্ন' করো
time = 0;                                // সময়কে ০ থেকে শুরু করো
completed_count = 0;                      // কাউন্টার শূন্য করো

```

```

cout << "\n\nFinal Gantt Chart:\n";
cout << "0"; // গ্যান্ট চার্ট শুরু সময় ০ দিয়ে

```

int last_process = -1; // আগের প্রসেস ট্র্যাক করার জন্য (যাতে বারবার নাম না প্রিন্ট হয়)

```

while(completed_count < n) {
    int idx = -1;
    int min_rt = 100000;

```

// আবার Shortest Remaining Time খুঁজে বের করা (একই লজিক)

```

for(int i=0; i<n; i++) {
    if(at[i] <= time && !completed[i]) {
        if(rt[i] < min_rt) {
            min_rt = rt[i];

```

```

        idx = i;
    }
}

if(idx != -1) {
    // যদি আগের প্রসেস আর বর্তমান প্রসেস একই না হয়, তবে নতুন ব্লক শুরু
    if(last_process != idx) {
        cout << " -- P" << (idx+1); // প্রসেসের নাম প্রিন্ট
        last_process = idx;
    }

    rt[idx]--; // ১ ইউনিট কাজ করো
    time++; // ১ ইউনিট সময় বাড়ে

    // যদি প্রসেস শেষ হয়ে যায়
    if(rt[idx] == 0) {
        completed[idx] = true;
        completed_count++;
        cout << " -- " << time; // প্রসেস শেষ হওয়ার সময় প্রিন্ট
        last_process = -1; // পরবর্তী প্রসেসের জন্য রিসেট
    }
} else {
    // CPU Idle
    cout << " -- Idle -- " << (time + 1);
    time++;
    last_process = -1;
}
}

```

৯. স্যারকে যা বলবে:

"স্যার, গ্যান্ট চার্টটি সুন্দরভাবে দেখানোর জন্য আমি একটি আলাদা লুপ ব্যবহার করেছি।

১. প্রথমে `rt`, `completed`, এবং `time` কে রিসেট করেছি যাতে আবার শুরু থেকে সিমুলেশন চালাতে পারি।
২. লুপের লজিক একই, কিন্তু এখানে প্রিন্টিংয়ের বিষয় আছে।
৩. `last_process` ভেরিয়েবল ব্যবহার করে চেক করছি যে প্রসেস পরিবর্তন হয়েছে কিনা। যদি পরিবর্তন হয়, তবে নতুন প্রসেসের নাম প্রিন্ট করছি।
৪. যখন প্রসেসটি সম্পূর্ণ শেষ হয় (`rt==0`), তখন শেষ সময় প্রিন্ট করছি। এতে Preemption সহ গ্যান্ট চার্টটি পরিষ্কারভাবে দেখা যায়।"

```

// --- আউটপুট টেবিল ---
cout << "\n\nProcess\tAT\tBT\tCT\tTAT\tWT\n";
for(int i=0; i<n; i++) {
    cout << "P" << (i+1) << "\t" << at[i] << "\t" << bt[i] << "\t"
        << ct[i] << "\t" << tat[i] << "\t" << wt[i] << endl;
}

// --- গড় মান ---
cout << "\nAverage Waiting Time: " << (float)total_wt / n << endl;
cout << "Average Turnaround Time: " << (float)total_tat / n << endl;

return 0;
}

```

□ স্যারকে যা বলবে: "শেষে, একটি টেবিলের মাধ্যমে প্রতিটি প্রসেসের বিস্তারিত তথ্য দেখানো হয়েছে। এবং মোট WT ও TAT কে প্রসেস সংখ্যা দিয়ে ভাগ করে গড় ওয়েটিং টাইম এবং গড় টার্নঅরাউন্ড টাইম বের করা হয়েছে। (float) ব্যবহার করা হয়েছে যাতে দশমিক মান সঠিক আসে।"

>>>>>>>>>>

□ স্যারকে যা বলবে (Step-by-Step Explanation):

১. ভূমিকা (Introduction)

"স্যার, এই কোডটি SRTF (Shortest Remaining Time First) বা Preemptive SJF শেডিউলিং অ্যালগরিদম ইমপ্লিমেন্ট করে। এর মূল বৈশিষ্ট্য হলো, CPU সবসময় সেই প্রসেসটিকে দেবে যার Remaining Time (বাকি সময়) সবচেয়ে কম। যদি মাঝপথে কোনো নতুন প্রসেস আসে যার বাকি সময় বর্তমান প্রসেসের চেয়ে কম, তবে বর্তমান প্রসেসকে থামিয়ে (Preempt) নতুন প্রসেসটিকে CPU দেওয়া হয়।"

২. ডাটা ইনিশিয়ালাইজেশন (Initialization)

"প্রথমে আমি `rt[]` (Remaining Time) অ্যারে নিয়েছি যা শুরুতে `bt[]` (Burst Time) এর সমান থাকে। একটি `completed[]` বুলিয়ান অ্যারে রাখা হয়েছে যা ট্র্যাক করবে কোন প্রসেস শেষ হয়েছে।"

৩. মেইন লুপ ও সিলেকশন লজিক (Main Loop & Selection)

"এরপর একটি `while` লুপ চালানো হয়েছে যতক্ষণ না সব প্রসেস শেষ হয়। লুপের প্রতিবার রাউন্ডে: ১. একটি `for` লুপ চালিয়ে চেক করা হয় কোন প্রসেসগুলো ইতিমধ্যে এসে গেছে (`AT <= current_time`) এবং শেষ হয়নি। ২. তাদের মধ্যে থেকে যার Remaining Time সবচেয়ে কম, সেটিকে বেছে নেওয়া হয় (`idx` এবং `min_rt` আপডেট করে)। ৩. যদি কোনো প্রসেস পাওয়া না যায় (`idx == -1`), তবে CPU Idle থাকে এবং সময় ১ ইউনিট বাড়ানো হয়।"

৪. এক্সিকিউশন ও প্রিএম্পশন (Execution & Preemption)

"যখন একটি প্রসেস পাওয়া যায়:

- আমরা তার Remaining Time ১ ইউনিট কমাইছি (`rt[idx]--`) এবং সময় ১ ইউনিট বাড়িয়ে নিচ্ছি (`time++`)।
- এটি প্রতি ১ ইউনিট সময় পরপর চেক করে, ফলে যদি মাঝপথে কোনো নতুন ছোট প্রসেস আসে, তবে তাৎক্ষণিকভাবে সেটি CPU পায়। এটাই হলো Preemption।

- যখন কোনো প্রসেসের `rt[idx] == 0` হয়, তখন সেটি সম্পূর্ণ শেষ হয়। তখন তার Completion Time (CT), Turnaround Time (TAT), এবং Waiting Time (WT) বের করা হয় এবং প্রসেসটিকে `completed` চিহ্নিত করা হয়।"

৫. গ্যান্ট চার্ট প্রিন্টিং (Gantt Chart Logic)

- "স্যার, গ্যান্ট চার্টটি সুন্দরভাবে দেখানোর জন্য আমি একটি আলাদা লুপ ব্যবহার করেছি। প্রথমে সব ভেরিয়েবল (`rt`, `completed`, `time`) রিসেট করে আবার শুরু থেকে সিমুলেশন চালানো হয়েছে।
- এখানে `last_process` ভেরিয়েবল ব্যবহার করে ট্র্যাক করা হয়েছে যে প্রসেস পরিবর্তন হয়েছে কিনা।
 - যদি প্রসেস পরিবর্তন হয়, তবে নতুন নাম প্রিন্ট করা হয়েছে।
 - আর যখন প্রসেসটি সম্পূর্ণ শেষ হয়, তখন শেষ সময় প্রিন্ট করা হয়েছে। এতে Preemption সহ গ্যান্ট চার্টটি পরিষ্কারভাবে দেখা যায়।"

৬. ফাইনাল আউটপুট (Final Output)

"শেষে, একটি টেবিলের মাধ্যমে প্রতিটি প্রসেসের AT, BT, CT, TAT, WT দেখানো হয়েছে। এবং মোট WT ও TAT কে প্রসেস সংখ্যা দিয়ে ভাগ করে গড় ওয়েটিং টাইম এবং গড় টার্নঅরাউন্ড টাইম বের করা হয়েছে। (`float`) ব্যবহার করা হয়েছে যাতে দশমিক মান সঠিক আসে।"

⚡ স্যারের সম্ভাব্য প্রশ্নের উত্তর (Quick Fire):

প্রশ্ন	উত্তর
SJF এবং SRTF এর পার্থক্য?	SJF Non-Preemptive (থামে না), SRTF Preemptive (মারপথে থামতে পারে)।
কেন <code>rt[]</code> অ্যারে লাগল?	কারণ প্রসেস মারপথে থামতে পারে, তাই বাকি সময় ট্র্যাক করতে <code>rt[]</code> লাগে।
Preemption কখন হয়?	যখন নতুন কোনো প্রসেস আসে যার Remaining Time বর্তমান প্রসেসের চেয়ে কম।
কেন দুইবার লুপ চালানো হয়েছে?	প্রথমবার শুধু CT/TAT/WT হিসাবের জন্য, দ্বিতীয়বার সুন্দর গ্যান্ট চার্ট প্রিন্ট করার জন্য।
(<code>float</code>) কেন ব্যবহার করেছেন?	গড় মানের দশমিক অংশ ঠিক রাখার জন্য।

Priority:NON-preemitive:

#include <iostream>


```

using namespace std;

int main() {

int n;

cout << "Enter number of processes: ";

cin >> n;

// অ্যােরে ডিক্লেয়ারেশন

int pid[10], at[10], bt[10], pr[10]; // pr = Priority

int ct[10], tat[10], wt[10];

bool done[10] = {false}; // কোন প্রসেস শেষ হয়েছে তা ট্র্যাক করার জন্য

// ১. ইনপুট নেওয়া

for(int i = 0; i < n; i++) {

pid[i] = i + 1;

cout << "\nEnter Arrival Time for P" << pid[i] << ": ";

cin >> at[i];

cout << "Enter Burst Time for P" << pid[i] << ": ";

cin >> bt[i];

cout << "Enter Priority for P" << pid[i] << " (Higher is better): ";

cin >> pr[i];

}

int time = 0; // বর্তমান সময়

int completed = 0; // কতগুলো প্রসেস শেষ হয়েছে

int total_wt = 0, total_tat = 0;

cout << "\nGantt Chart:\n";

cout << time; // শুরু সময় (সাধারণত 0)

// ২. মেইন লুপ (যতক্ষণ না সব প্রসেস শেষ হয়)

```

```

while(completed < n) {

int idx = -1; // কোন প্রসেস নেওয়া হবে

int max_priority = -1; // সবচেয়ে বেশি প্রায়োরিটি খুঁজার জন্য (-1 দিয়ে শুরু)

// রেডি কিউ থেকে Highest Priority প্রসেস খুঁজুন

for(int i = 0; i < n; i++) {

// শর্ত ১: প্রসেস এসেছে? (AT <= time)

// শর্ত ২: প্রসেস শেষ হয়নি? (!done)

if(at[i] <= time && !done[i]) {

// শর্ত ৩: এর প্রায়োরিটি কি আগের সবচেয়ে বেশিটার চেয়েও বেশি?

if(pr[i] > max_priority) {

max_priority = pr[i]; // নতুন সর্বোচ্চ প্রায়োরিটি আপডেট

idx = i; // এই প্রসেসের ইনডেক্স সেভ করো

}

}

}

if(idx != -1) {

// প্রসেস পাওয়া গেছে

// গ্যান্ট চার্টে প্রসেসের নাম প্রিন্ট

cout << " -- P" << (idx+1) << " -- ";

time += bt[idx]; // সময় বাড়ান (পুরো বাস্ট টাইম যোগ করে)

ct[idx] = time; // Completion Time

tat[idx] = ct[idx] - at[idx]; // Turnaround Time

wt[idx] = tat[idx] - bt[idx]; // Waiting Time

total_wt += wt[idx];

```

```

total_tat += tat[idx];

done[idx] = true; // প্রসেস শেষ হিসেবে চিহ্নিত করুন

completed++;

cout << time; // শেষ সময় প্রিন্ট করুন

}

else {

// CPU Idle (কোনো প্রসেস পাওয়া যায়নি)

cout << " -- Idle -- ";

time++; // সময় ১ ইউনিট বাড়ান

cout << time; // নতুন সময় প্রিন্ট করুন

}

}

// ৩. আউটপুট টেবিল

cout << "\n\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n";

for(int i = 0; i < n; i++) {

cout << "P" << pid[i] << "\t"

<< at[i] << "\t"

<< bt[i] << "\t"

<< pr[i] << "\t"

<< ct[i] << "\t"

<< tat[i] << "\t"

<< wt[i] << endl;

}

// ৪. গড় মান

cout << "\nAverage Waiting Time: " << (float)total_wt / n << endl;

```

```
cout << "Average Turnaround Time: " << (float)total_tat / n << endl;

return 0;

}
```

১. **সময়**

“স্যার, এই কোডটি Non-Preemptive Priority Scheduling অ্যালগরিদম ইমপ্লিমেন্ট করে। এই অ্যালগরিদমে CPU সেই প্রসেসটিকে দেওয়া হয় যার **প্রায়োরিটি নাম্বার সবচেয়ে বেশি** (Higher number = Higher priority)। একবার একটি প্রসেস CPU পেলে, সেটি সম্পূর্ণ শেষ না হওয়া পর্যন্ত CPU ছাড়বে না।”

২. ইনপুট সেকশন (Input Section)

“প্রথমে ইউজার থেকে প্রসেসের সংখ্যা n ইনপুট নেওয়া হয়েছে। এরপর প্রতিটি প্রসেসের Arrival Time (AT), Burst Time (BT) এবং Priority (PR) ইনপুট নেওয়া হয়েছে। `done[]` অ্যারে রাখা হয়েছে যা ট্র্যাক করবে কোন প্রসেস শেষ হয়েছে।”

৩. **সময়**

“এরপর একটি `while` লুপ চালানো হয়েছে যতক্ষণ না সব প্রসেস শেষ হয়। লুপের প্রতিবার রাউন্ডে: ১. একটি `for` লুপ চালিয়ে চেক করা হয় কোন প্রসেসগুলো ইতিমধ্যে এসে গেছে ($AT \leq current_time$) এবং শেষ হয়নি। ২. তাদের মধ্যে থেকে যার Priority **নাম্বার সবচেয়ে বেশি** ($pr[i] > max_priority$), সেটিকে বেছে নেওয়া হয়। (নোট: SJF এ আমরা কম Burst Time খুঁজতাম, কিন্তু এখানে বেশি Priority নাম্বার খুঁজছি)।”

৪. **সময়**

“যখন একটি প্রসেস পাওয়া যায় ($idx \neq -1$):

- তার Burst Time অনুযায়ী `time` বাড়ানো হয়।
- Completion Time (CT), Turnaround Time (TAT), এবং Waiting Time (WT) বের করা হয়।
- প্রসেসটিকে `done[]` অ্যারেতে `true` করে দেওয়া হয়।

আর যদি কোনো প্রসেস পাওয়া না যায় (সবাই আসেনি), তবে CPU Idle থাকে এবং সময় ১ ইউনিট বাড়িয়ে আবার চেক করা হয়।”

৫. **সময়**

“গ্যান্ট চার্টটি লুপের মধ্যেই প্রিন্ট করা হয়েছে, যাতে Idle Time সহ প্রসেসের ক্রম দেখা যায়। শেষে একটি টেবিলের মাধ্যমে প্রতিটি প্রসেসের AT, BT, Priority, CT, TAT, WT দেখানো হয়েছে এবং গড় ওয়েটিং টাইম ও টার্নঅরাউন্ড টাইম বের করা হয়েছে।”

৬. **উত্তর**

৭. **গ্যান্ট চার্ট ও আউটপুট (Gantt Chart & Output)**

এটি কোন অ্যালগরিদম?	Non-Preemptive Priority Scheduling।
কিভাবে প্রসেস সিলেক্ট করা হয়?	যে প্রসেসের Priority নাম্বার সবচেয়ে বেশি এবং সে ইতিমধ্যে এসে গেছে, তাকে সিলেক্ট করা হয়।
Preemptive নাকি Non-Preemptive?	Non-Preemptive। একবার শুরু হলে শেষ পর্যন্ত চলে।
SJF এবং Priority এর পার্থক্য?	SJF এ কম Burst Time দেখা হয়, Priority তে বেশি Priority নাম্বার দেখা হয়।
যদি দুটি প্রসেসের প্রায়োরিটি সমান হয়?	সাধারণত যে আগে আসে (Lower Arrival Time) বা ইনপুটে আগে আছে, সে আগে পায় (FCFS হিসেবে কাজ করে)।
Idle Time কখন হয়?	যখন বর্তমান সময়ে কোনো প্রসেস উপস্থিত নেই (সবাই পরে আসবে)।

////////



1. `int max_priority = -1;` শুরুতে প্রায়োরিটি -1 ধরা হয়েছে যাতে যেকোনো পজিটিভ প্রায়োরিটি নাম্বার এর চেয়ে বড় হয়।
2. `if(pr[i] > max_priority):` এখানে > ব্যবহার করা হয়েছে কারণ Higher Number = Higher Priority। (SJF এ < ব্যবহার হতো)।
3. `time += bt[idx];` Non-Preemptive হওয়ার কারণে পুরো Burst Time যোগ করে সময় বাড়ানো হচ্ছে।

এই সামারিটি দিয়ে তুমি খুব সহজেই স্যারকে বুঝিয়ে দিতে পারবে! শুভকামনা! □
 bhai code gula sho short e bolo konta ki sir k bujhanor jnno atto boro deo na portevalo
 lage na

ভাই, ঠিক আছে। ল্যাব ভাইভা বা প্রেজেন্টেশনের জন্য খুব ছোট এবং পয়েন্ট আকারে সামারি নিচে দেওয়া হলো। স্যারের সামনে দাঁড়িয়ে এই ৪-৫টি পয়েন্ট বললেই হবে।



1. অ্যালগরিদম: "স্যার, এটি Non-Preemptive Priority Scheduling। অর্থাৎ, যার প্রায়োরিটি নাম্বার বেশি, সে আগে CPU পাবে। একবার শুরু হলে শেষ পর্যন্ত চলে।"
2. সিলেকশন লজিক: "আমরা একটি লুপ চালিয়ে দেখি কোন প্রসেসগুলো এসে গেছে ($AT \leq time$)। তাদের মধ্যে থেকে যার Priority সবচেয়ে বেশি ($max_priority$), সেটিকে বেছে নেই।"
3. Idle Time: "যদি বর্তমান সময়ে কোনো প্রসেস না থাকে (সবাই পরে আসবে), তবে CPU Idle থাকে এবং সময় ১ ইউনিট বাড়িয়ে আবার চেক করি।"



4. **হিসাব:** "প্রসেস শেষ হলে তার CT, TAT এবং WT বের করে টেবিলে দেখাই এবং গড় মান বের করি।"

1. `if(pr[i] > max_priority):` (সবচেয়ে গুরুত্বপূর্ণ) এখানে চেক করছি কার প্রায়োরিটি বেশি।
2. `time += bt[idx]:` পুরো কাজ শেষ করে তবেই সময় বাড়ানো হচ্ছে (Non-Preemptive)।
3. `done[idx] = true:` একবার শেষ হলে আর নেওয়া যাবে না।

র

□□
□□
□□

Priority Preemptive :

#include <iostream>

□□
□□
□□

using namespace std;

int main() {

□□
□□
□□

int n;

□□
□□
□□

cout << "Enter number of processes: ";

if (n >> n;

□□
□□
□□

int at[10], bt[10], pr[10]; // Arrival, Burst, Priority

□□
□□
□□

int rt[10]; // Remaining Time

□□
□□
□□

int ct[10], tat[10], wt[10];

□□
□□
□□

bool completed[10] = {false};

□□
□□
□□

// Input

□□
□□
□□

for (int i = 0; i < n; i++) {

cout << "\nEnter Arrival Time for P" << (i+1) << ": ";

cin >> at[i];

□□
□□
□□

cout << "Enter Burst Time for P" << (i+1) << ": ";

□□
□□
□□

কোন):

cin >> bt[i];

cout << "Enter Priority for P" << (i+1) << " (Higher is better): ";

```

cin >> pr[i];

rt[i] = bt[i]; // Initially Remaining Time = Burst Time

}

int time = 0;

int completed_count = 0;

int total_wt = 0, total_tat = 0;

cout << "\nGantt Chart:\n";

cout << time;

int last_process = -1; // To track process changes for Gantt Chart

while(completed_count < n) {

int idx = -1;

int max_priority = -1; // Start with lowest possible priority

// Find process with Highest Priority among arrived & incomplete processes

for(int i = 0; i < n; i++) {

if(at[i] <= time && !completed[i]) {

if(pr[i] > max_priority) {

max_priority = pr[i];

idx = i;

}

}

}

if(idx != -1) {

// If process changed, print new block in Gantt Chart

if(last_process != idx) {

cout << " -- P" << (idx+1);

```

```

last_process = idx;

}

rt[idx]--; // Execute for 1 unit

time++; // Move time forward

if(rt[idx] == 0) {

    completed[idx] = true;

    completed_count++;

    ct[idx] = time;

    tat[idx] = ct[idx] - at[idx];

    wt[idx] = tat[idx] - bt[idx];

    total_wt += wt[idx];

    total_tat += tat[idx];

    cout << " -- " << time; // Print completion time

    last_process = -1; // Reset for next process

}

} else {

    // CPU Idle

    cout << " -- Idle -- " << (time + 1);

    time++;

    last_process = -1;

}

}

// Output Table

cout << "\n\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n";

for(int i = 0; i < n; i++) {

```



```

cout << "P" << (i+1) << "\t"

<< at[i] << "\t"

<< bt[i] << "\t"

<< pr[i] << "\t"

<< ct[i] << "\t"

<< tat[i] << "\t"

<< wt[i] << endl;

}

cout << "\nAverage Waiting Time: " << (float)total_wt / n << endl;

cout << "Average Turnaround Time: " << (float)total_tat / n << endl;

return 0;

}

```

🔑 মূল পার্থক্য (Non-Preemptive vs Preemptive):

1. **Non-Preemptive:** একবার প্রসেস নিলে `time += bt[idx]` করে পুরোটা শেষ করতাম।
2. **Preemptive:** এখানে `rt[idx]--` এবং `time++` করে প্রতি ১ ইউনিট সময়ে চেক করছি নতুন কোনো উচ্চ প্রাধান্যের (Higher Priority) প্রসেস এসেছে কিনা। যদি আসে, তবে `idx` পরিবর্তন হয়ে যাবে এবং নতুন প্রসেসটি CPU পাবে।

এই কোডটি রান করলে প্রিগ্রস্পশন সহ সঠিক গ্যান্ট চার্ট পাবে! 😊

□ স্যারকে যা বলবে (Step-by-Step Explanation):

১. ভূমিকা (Introduction)

"স্যার, এই কোডটি **Preemptive Priority Scheduling** অ্যালগরিদম ইমপ্লিমেন্ট করে। এখানে 'Higher Number = Higher Priority' ধরা হয়েছে। এর মূল বৈশিষ্ট্য হলো, CPU সবসময় সেই প্রসেসটিকে দেবে যার প্রায়োরিটি নাম্বার সবচেয়ে বেশি। যদি মাঝপথে কোনো নতুন প্রসেস আসে যার প্রায়োরিটি বর্তমান প্রসেসের চেয়ে বেশি, তবে বর্তমান প্রসেসকে থামিয়ে (Preempt) নতুন প্রসেসটিকে CPU দেওয়া হয়।"

২. মেইন লজিক: প্রতি ১ ইউনিট সময়ে চেক করা (The Core Logic)

"কোডটিতে একটি `while` লুপ চালানো হয়েছে যতক্ষণ না সব প্রসেস শেষ হয়। লুপের প্রতিবার রাউন্ডে: ১. একটি `for` লুপ চালিয়ে চেক করা হয় কোন প্রসেসগুলো ইতিমধ্যে এসে গেছে (`AT <= current_time`) এবং শেষ হয়নি। ২. তাদের মধ্যে থেকে যার `Priority` নাম্বার সবচেয়ে বেশি (`pr[i] > max_priority`), সেটিকে বেছে নেওয়া হয়। ৩. নির্বাচিত প্রসেসটির `Remaining Time (rt)` ১ ইউনিট কমানো হয় এবং সময় ১ ইউনিট বাড়ানো হয়।"

৩. প্রিএম্পশন কিভাবে কাজ করে? (How Preemption Works)

"যেহেতু আমরা প্রতি ১ ইউনিট সময় পরপর আবার `for` লুপ চালিয়ে সেরা প্রসেস খুঁজছি, তাই যদি মাঝপথে কোনো নতুন প্রসেস আসে যার প্রায়োরিটি বর্তমান প্রসেসের চেয়ে বেশি, তবে `idx` পরিবর্তন হয়ে যাবে এবং নতুন প্রসেসটি CPU পাবে। এটাই হলো **Preemption**।"

৪. প্রসেস সম্পন্ন হওয়া (Completion)

"যখন কোনো প্রসেসের `Remaining Time` শূন্য (`rt[idx] == 0`) হয়ে যায়, তখন সেটি সম্পূর্ণ শেষ হয়। তখন তার `Completion Time (CT)`, `Turnaround Time (TAT)`, এবং `Waiting Time (WT)` বের করা হয় এবং প্রসেসটিকে `completed` চিহ্নিত করা হয়।"

৫. গ্যান্ট চার্ট এবং আউটপুট (Gantt Chart & Output)

"গ্যান্ট চার্টটি সুন্দরভাবে দেখানোর জন্য `last_process` ভেরিয়েবল ব্যবহার করা হয়েছে। যদি প্রসেস পরিবর্তন হয়, তবে নতুন নাম প্রিন্ট করা হয়। আর যখন প্রসেসটি সম্পূর্ণ শেষ হয়, তখন শেষ সময় প্রিন্ট করা হয়। শেষে টেবিল এবং গড় মান প্রিন্ট করা হয়েছে।"

⚡ স্যারের সম্ভাব্য প্রশ্নের উত্তর (Quick Fire):

প্রশ্ন	উত্তর
Non-Preemptive vs Preemptive Priority?	Non-Preemptive এ একবার শুরু হলে শেষ পর্যন্ত চলে। Preemptive এ মাঝপথে উচ্চ প্রাধান্যের প্রসেস আসলে থামিয়ে নতুনটা নেয়।
কেন <code>rt[]</code> (Remaining Time) লাগল?	কারণ প্রসেস মাঝপথে থামতে পারে, তাই বাকি সময় ট্র্যাক করতে <code>rt[]</code> লাগে।
প্রিএম্পশন কখন হয়?	যখন নতুন কোনো প্রসেস আসে যার <code>Priority</code> নাম্বার বর্তমান প্রসেসের চেয়ে বেশি।
কেন দুইবার লুপ	না স্যার, এখানে মূল লজিক একবারই চলছে। গ্যান্ট চার্ট

চালানো হয়েছে?	প্রিন্টিং একই লুপের মধ্যে করা হয়েছে যাতে কোড সহজ থাকে। (নোট: আগের SRTF কোডে দুইবার ছিল, কিন্তু এখানে একবারেই করা হয়েছে সহজ করার জন্য)।
(float) কেন ব্যবহার করেছেন?	গড় মানের দশমিক অংশ ঠিক রাখার জন্য।

✓কোডের গুরুত্বপূর্ণ লাইনগুলোর ব্যাখ্যা (For Deep Understanding):

1. `if(pr[i] > max_priority):` (সবচেয়ে গুরুত্বপূর্ণ) এখানে চেক করছি কার প্রায়োরিটি বেশি।
2. `rt[idx]--` এবং `time++`: প্রতি ১ ইউনিট সময়ে কাজ কমিয়ে সময় বাড়ানো হচ্ছে, যাতে পরবর্তী রাউন্ডে নতুন প্রসেস আসলে তা ধরা পড়ে।
3. `if(last_process != idx):` গ্যান্ট চার্টে বারবার একই প্রসেসের নাম প্রিন্ট এড়াতে এটি ব্যবহার করা হয়েছে।

এই সামারিটি দিয়ে তুমি খুব সহজেই স্যারকে বুঝিয়ে দিতে পারবে যে কোডটি কিভাবে কাজ করছে! শুভকামনা! ☐

????

Priority (lower num higher priority)preemptive

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    int at[10], bt[10], pr[10];
    int rt[10];
    int ct[10], tat[10], wt[10];
    bool completed[10] = {false};

    for(int i = 0; i < n; i++) {
        cout << "\nEnter Arrival Time for P" << (i+1) << ": ";
        cin >> at[i];
        cout << "Enter Burst Time for P" << (i+1) << ": ";
        cin >> bt[i];
        cout << "Enter Priority for P" << (i+1) << " (Lower is better): ";
        cin >> pr[i];

        rt[i] = bt[i];
    }

    int time = 0;
    int completed_count = 0;
    int total_wt = 0, total_tat = 0;
```

```

cout << "\nGantt Chart:\n";
cout << time;

int last_process = -1;

while(completed_count < n) {
    int idx = -1;
    int min_priority = 100000;

    for(int i = 0; i < n; i++) {
        if(at[i] <= time && !completed[i]) {
            if(pr[i] < min_priority) {
                min_priority = pr[i];
                idx = i;
            }
        }
    }

    if(idx != -1) {
        if(last_process != idx) {
            cout << " -- P" << (idx+1);
            last_process = idx;
        }

        rt[idx]--;
        time++;

        if(rt[idx] == 0) {
            completed[idx] = true;
            completed_count++;

            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];

            total_wt += wt[idx];
            total_tat += tat[idx];

            cout << " -- " << time;
            last_process = -1;
        }
    } else {
        cout << " -- Idle -- " << (time + 1);
        time++;
        last_process = -1;
    }
}

cout << "\n\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n";
for(int i = 0; i < n; i++) {
    cout << "P" << (i+1) << "\t"
        << at[i] << "\t"
        << bt[i] << "\t"
        << pr[i] << "\t"
        << ct[i] << "\t"
        << tat[i] << "\t"
        << wt[i] << endl;
}

```

```

}

cout << "\nAverage Waiting Time: " << (float)total_wt / n << endl;
cout << "Average Turnaround Time: " << (float)total_tat / n << endl;

return 0;
}

```

✓ Non-Preemptive Priority Code (Lower Number = Higher Priority):

```

#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter number of processes: ";
    cin >> n;

    int pid[10], at[10], bt[10], pr[10];
    int ct[10], tat[10], wt[10];
    bool done[10] = {false};

    for(int i = 0; i < n; i++) {
        pid[i] = i + 1;
        cout << "\nEnter Arrival Time for P" << pid[i] << ": ";
        cin >> at[i];

        cout << "Enter Burst Time for P" << pid[i] << ": ";
        cin >> bt[i];

        cout << "Enter Priority for P" << pid[i] << " (Lower is better): ";
        cin >> pr[i];
    }

    int time = 0;
    int completed = 0;
    int total_wt = 0, total_tat = 0;

    cout << "\nGantt Chart:\n";
    cout << time;

    while(completed < n) {
        int idx = -1;
        int min_priority = 100000; // Lower number is better, so start with high value

        // Find process with Lowest Priority Number among arrived & incomplete processes
        for(int i = 0; i < n; i++) {
            if(at[i] <= time && !done[i]) {
                if(pr[i] < min_priority) {
                    min_priority = pr[i];
                    idx = i;
                }
            }
        }

        // Process execution
        time += bt[idx];
        wt[idx] = time - at[idx];
        tat[idx] = time;
        total_wt += wt[idx];
        total_tat += tat[idx];
        done[idx] = true;
        completed++;
    }

    cout << "\nGantt Chart:\n";
    cout << time;
}

```

```

if(idx != -1) {
    cout << " -- P" << (idx+1) << " -- ";

    time += bt[idx];    // Execute full burst time (Non-Preemptive)
    ct[idx] = time;
    tat[idx] = ct[idx] - at[idx];
    wt[idx] = tat[idx] - bt[idx];

    total_wt += wt[idx];
    total_tat += tat[idx];

    done[idx] = true;
    completed++;

    cout << time;
}
else {
    cout << " -- Idle -- ";
    time++;
    cout << time;
}
}

cout << "\n\nProcess\tAT\tBT\tPR\tCT\tTAT\tWT\n";
for(int i = 0; i < n; i++) {
    cout << "P" << pid[i] << "\t"
        << at[i] << "\t"
        << bt[i] << "\t"
        << pr[i] << "\t"
        << ct[i] << "\t"
        << tat[i] << "\t"
        << wt[i] << endl;
}

cout << "\nAverage Waiting Time: " << (float)total_wt / n << endl;
cout << "Average Turnaround Time: " << (float)total_tat / n << endl;

return 0;
}

```